RSU · Basics of AI and Machine Learning · April 2026

Credit Card Fraud Detection with Machine Learning

Building a complete ML pipeline - from 340,000 raw transactions to a trained model that catches fraud in real time.

Explore the Project ↓

THE PROBLEM

Why Fraud Detection Is Hard

Card fraud costs the industry **billions of dollars every year**. Detection systems are the last line of defence between criminals and customers — and a single missed transaction can wipe out a customer's savings overnight.

New credit card company, western US. Main goal:

Catch as much fraud as possible - before the customer loses money.

> *"When in doubt - stop the transaction.*
> *A blocked card can be unblocked in minutes.*
> ***Money stolen is gone.***"

The Accuracy Trap

Only **0.52%** of transactions are fraud.

99.5%

accurate - catches **ZERO** fraud

A model that always says "not fraud" scores 99.5% - and is completely useless. Standard accuracy cannot be trusted here.

Two Types of Mistakes

Not all errors are equal - one is far more costly

FALSE POSITIVE

Legit → Flagged

A real purchase is blocked. The customer calls the bank, card is unblocked, customer moves on.

Cost: minor inconvenience

FALSE NEGATIVE ⚠

Fraud → Missed

Real fraud slips through. The customer loses money. The bank pays chargebacks, loses trust, and faces regulatory scrutiny.

Cost: financial loss + reputation damage

These two types of mistakes have very different costs - and that shapes every decision in the project.

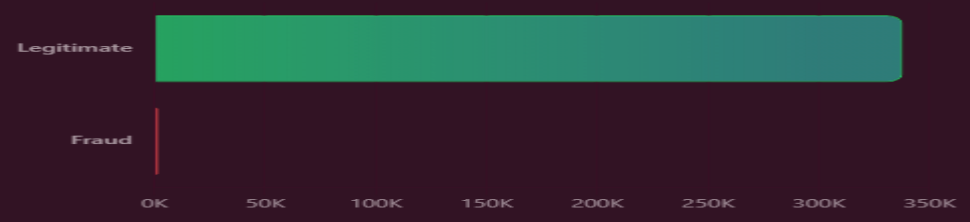
THE DATA

339,607 Real Transactions

Credit card transactions from the western United States. Each row is one purchase - customer details, merchant, category, location, amount, timestamp, and whether it was **fraud**.

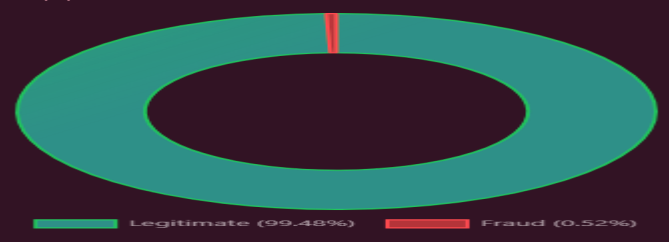
339,607	15	1,782	0.52%
Total Transactions	Original Columns	Fraud Cases	Fraud Rate

Volume - Legitimate vs Fraud



337,825 legitimate transactions vs 1,782 fraud

Share of Dataset (%)



99.48% legitimate | **0.52% fraud**

The Core Challenge

Only 1 in 193 transactions is fraud.

> *This imbalance shapes every decision that follows — **how we train, how we balance the data, and how we measure success.***

DATA FINDINGS

What Does Fraud Look Like?

Core Finding

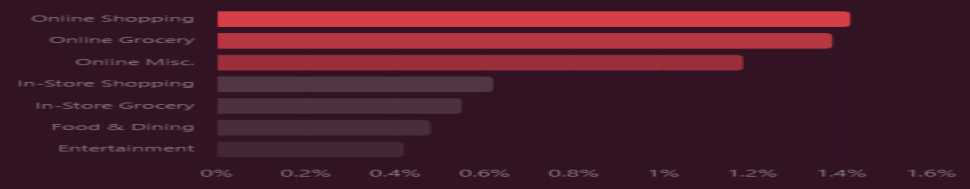
Fraud has a fingerprint.

> *Night hours, large amounts, distant merchants — **we found it before training a single model.***

Fraud Rate by Hour of Day

Fraud peaks between **1–4 AM** - criminals act while victims sleep.

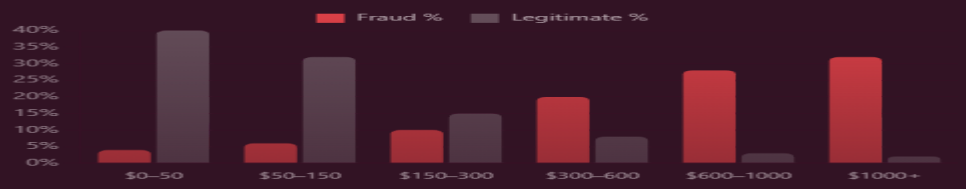
Top Fraud Categories



Online categories (**Online Shopping**, **Online Misc.**) have elevated fraud rates - no physical card required.

Transaction Amount Distribution

Fraud transactions cluster in higher amount ranges — average **$518** vs **~$70** legitimate



Distance from Cardholder Home (km)

Fraud happens **far from home** — ranked **#3** in feature importance (behind amt and age)

Transaction Amount

Fraud transactions are significantly larger than legitimate ones. Criminals go for big purchases.

$518
avg fraud

~$70
avg legitimate

Geographic Distance

Fraudulent merchants are located much further from the cardholder's home — often in a different state or country.

Distance ranked #3 in feature importance

Engineered from raw GPS coordinates using the

Four Signals, One Fingerprint

Every pattern we found became a feature we built:

Hour — captures the 1–4 AM peak

Amount — unusually large purchases

Distance — far from home

FEATURE ENGINEERING

Turning Raw Data Into Fraud Signals

The Idea

The raw data doesn't speak fraud. We taught it how.

> *A timestamp is just a number. But **3 AM on a Wednesday** tells a very different story.*

Time of Day

A raw timestamp is meaningless to a model. We extracted **hour**, **day of week**, and **month** — so the model can learn that 3 AM is suspicious and noon is not.

`2019-01-01 03:24:00` → `hour = 3` ⚠

Distance to Merchant

The dataset has the cardholder's home coordinates and the merchant's location. We calculated the straight-line distance between them. A purchase **4,000 km from home** is a major red flag. This became our **top predictor.**

`lat/lon × 2 pairs` → `distance = 4,823 km` ⚠

Customer Age

A date of birth means nothing on its own. We calculated each cardholder's age at the time of the transaction. Fraud rates differ significantly across age groups — older cardholders are targeted more often.

`dob: 1968-03-14` → `age = 51`

Amount — Compressed Scale

Transaction amounts range from $1 to over $28,000. That extreme range confuses some models — a $28,000 purchase would dominate all other signals. Taking the **logarithm** brings everything into a manageable range without losing the pattern.

`amount: $28,476` → `log_amt = 10.25`

Categories → Yes / No Columns

A model cannot read the word "*Online Shopping*". So we created one column per category — each asking a yes/no question the model can actually process.

`is_online_shopping?` `is_food_dining?` `is_entertainment?` + 11 more

Same applied to 13 US states. This technique is called **one-hot encoding**.

The outcome

15 → **39**

raw columns — engineered features

4 of our hand-crafted features ranked among the model's **top predictors**

DATA PREPARATION

Splitting the Data — Before Any Model Sees It

The Rule

You can't test a student on questions they've already seen the answers to.

> *We locked away 20% of the data before any training happened — **the model never saw it until the final exam.***

The 80 / 20 Split

80% — Training Data (271,685 transactions) | **20% — Test**

TRAINING SET

What the model **learns from**. It sees these transactions during training and adjusts itself to detect patterns.

TEST SET — LOCKED AWAY

The **real exam**. The model never sees these during training. All final results — Recall, F1, AUC — are measured here.

reproducibility `random_state=42`

preserves fraud ratio `stratify=y`

Fixing the Imbalance — SMOTE

Generates synthetic fraud examples by blending existing ones — until the training set is balanced. The model now sees equal amounts of both classes.

0.52% fraud before SMOTE → **50%** fraud after SMOTE

⚠ Applied **only to the training set** — test set stays untouched.

Putting Features on Equal Footing

StandardScaler rescales all numeric features to the same centre and spread — so a feature with large numbers doesn't dominate one with small numbers.

`amount: $0 – $28,476` → `scaled: -1.2 to 3.4`

⚠ Fitted **on training data only** — prevents test information from leaking into training.

EVALUATION

How We Measure Success

Why Not Accuracy?

A model that always says "not fraud" would be 99.5% accurate — and catch zero fraud.

> *We need metrics that focus specifically on the fraud class — **not on how often we get the easy 99.5% right.***

R Recall

Of all the real fraud cases — how many did we catch?

Example

356 fraud cases in test set → model catches **299** → Recall = **0.84**

★ Our primary metric

F F1-Score

Balances catching fraud against over-flagging legitimate transactions.

The trade-off

Flag everything → perfect Recall, zero Precision → **F1 keeps both in check**

Used to tune model thresholds

A ROC-AUC

How well the model separates fraud from legitimate across all thresholds.

Score range

0.5 = random guessing → **1.0** = perfect separation

Our best model reached 0.9978

MODEL TRAINING

5 Algorithms, From Simple to Powerful

The Approach

Start simple. Add complexity only when needed.

> *Each model fixes the weakness of the one before it — **so we always know exactly what improvement each step buys us.***

1 Logistic Regression

Draws a **single straight line** between fraud and not-fraud. Fast and easy to explain — our starting point to measure all other models against.

+ Fast, simple, easy to explain to anyone
- Fraud patterns are not a straight line

2 Decision Tree

A flowchart of **yes/no questions**: "Is amount > $500? Is the hour after midnight? Is the distance > 1,000 km?" Can capture complex combinations of signals.

+ Can learn complex, non-linear fraud patterns
- Memorises training data — performs poorly on new transactions

3 Random Forest

Grows **hundreds of trees**, each on a random slice of the data, then lets them all vote. No single tree dominates — the crowd is more reliable than any individual expert.

+ Voting across many trees eliminates overfitting
- Trees are independent — they don't learn from each other's mistakes

GRADIENT BOOSTING — WHERE EACH TREE FIXES THE LAST ONE'S MISTAKES

4 XGBoost

Builds trees **one at a time in sequence**. Each new tree looks specifically at what the previous trees got wrong and tries to fix those cases. Like a team of editors — each person only corrects what the last one missed.

+ Highest fraud catch rate of all five models
- More false alarms — flags more legitimate transactions

5 LightGBM DEPLOYED ★

Same sequential idea as XGBoost, but engineered by Microsoft for **speed and large datasets**. Chosen for deployment because it catches nearly as much fraud as XGBoost — while producing far fewer false alarms.

+ Fewer false alarms, faster inference on large data
- Catches slightly less fraud than XGBoost

EVALUATION

Model Performance on Unseen Data

The Real Exam

All numbers come from data the models never saw.

> *The test set was locked away before training began — **this is the only number that matters.***

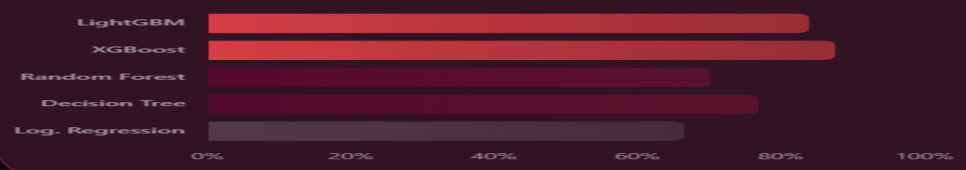
Model	Accuracy	Precision	Recall	F1-Score	ROC-AUC
LightGBM ★	0.9988	0.9257	0.8399	0.8807	0.9978
XGBoost	0.9987	0.8691	0.8764	0.8727	0.9964
Random Forest	0.9978	0.8475	0.7022	0.7680	0.9903
Decision Tree	0.9965	0.6343	0.7697	0.6954	0.8837
Logistic Regression	0.9888	0.2699	0.6657	0.3841	0.8767

Fraud Cases Caught vs Missed — out of 356 in the test set

The most concrete way to see model performance — actual fraud cases, not percentages

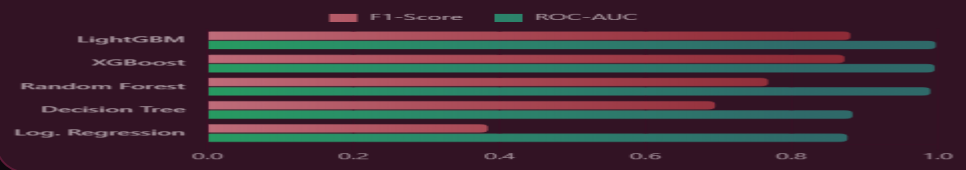
Recall — % of Fraud Caught

Higher is better. Gradient boosting pulls clearly ahead.



F1-Score & ROC-AUC

F1 balances precision and recall. AUC measures overall separation.



Confusion Matrix — LightGBM ★

Every transaction ends up in one of four boxes

MODEL SAID FRAUD

MODEL SAID LEGIT

Feature Importance — LightGBM

How much each feature influenced the model's fraud decisions

After training, we asked the model: "*which signals did you rely on most?*" A higher bar means the model

THRESHOLD OPTIMISATION

Tuning When the Model Says "Fraud"

The Idea

The model outputs a probability. We decide the cutoff.

> *Default is 0.5 — but lowering it catches more fraud at the cost of more false alarms.* ***The business chooses where on that curve to sit.***

Threshold 0.7 — Cautious

Only flags very high-confidence fraud. Fewer false alarms, but misses some real fraud.

Recall **82.3%** · Precision **94.2%**

293 caught · 18 false alarms

Threshold 0.4 — Optimal F1 ★

Best balance between catching fraud and keeping false alarms low. Selected as our optimal point.

Recall **85.4%** · Precision **91.6%**

304 caught · 28 false alarms

Threshold 0.1 — Maximum Recall

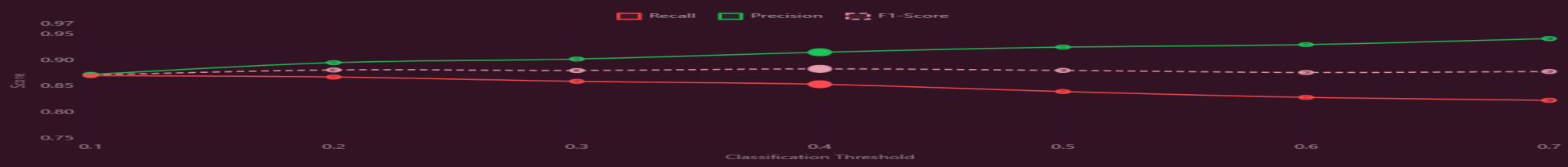
Catches almost everything — but raises far more false alarms. Customers get blocked more often.

Recall **87.1%** · Precision **87.3%**

310 caught · 45 false alarms

Precision / Recall / F1 vs Threshold — LightGBM

As threshold increases, Recall drops and Precision rises. F1 peaks at 0.4 — the sweet spot.



All Thresholds — Full Results

Threshold	Recall	Precision	F1-Score	Fraud Caught	False Alarms
0.1	0.8708	0.8732	0.8720	310	45
0.2	0.8680	0.8957	0.8816	309	36
0.3	0.8596	0.9027	0.8806	306	33

BUSINESS IMPACT

What Model Choice Means in Dollars

Bottom Line

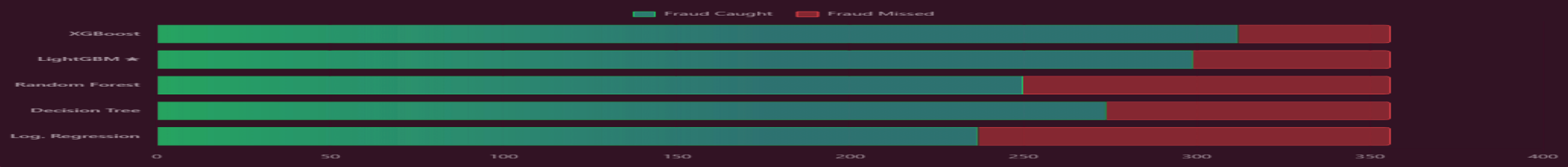
Choosing the right model prevents up to $39,000 in fraud losses — on this test set alone.

> *Scale that to millions of real transactions —* ***and the gap between a good model and a bad one becomes enormous.***

$518	312	299	237
Avg. Fraud Transaction	Caught — XGBoost (most fraud stopped)	Caught — LightGBM (deployed, best balance)	Caught — Logistic Regression (worst)

Estimated Fraud Loss — Per Model

Missed fraud cases × $518 average transaction value



The Cost of the Wrong Model

Logistic Regression misses **119 fraud cases** — an estimated loss of **$61,650** on this test set. It also raises 641 false alarms. The worst of both worlds.

Why We Deployed LightGBM, Not XGBoost

XGBoost catches **13 more fraud cases** (~$6,700 extra saved) but raises **47 false alarms**. LightGBM catches 299 cases with only **24 false alarms** — the best overall balance of fraud caught and customers not wrongly blocked.

KEY TAKEAWAYS

Five Things to Remember

The Bottom Line

Good machine learning is not about the algorithm. It's about the decisions made before and after it.

> *The right features, the right metric, the right evaluation — **those choices matter more than which model you pick.***

1 **Accuracy is meaningless on imbalanced data**

When 99.5% of your data is one class, any model that ignores the minority looks great. Always evaluate with Recall, F1, and ROC-AUC when classes are unbalanced.

2 **Feature engineering matters as much as model choice**

Hour, distance, age, and log amount — features we built from scratch — ranked among the top predictors. A well-crafted feature beats a fancier algorithm on poor features every time.

3 **Gradient boosting dominates tabular fraud detection**

XGBoost and LightGBM outperformed every simpler model by a clear margin. Each tree in the sequence learns from the previous one's mistakes — that iterative focus is what captures subtle multi-feature fraud patterns.

4 **Data leakage is the most dangerous silent mistake**

SMOTE only on training data. Scaler fitted only on training data. Apply these to all data first and your metrics look better in the notebook — then fall apart in production. We were careful to avoid it.

5 **Model selection is a business decision, not a technical one**

The difference between our best and worst model is approximately $39,000 in prevented fraud on this test set. Scale that to millions of transactions and the stakes are enormous. The people choosing the model need to understand what Recall actually means.

LIVE DEMONSTRATION

See the Model in Action

The trained LightGBM model is deployed as a Flask API. A Next.js dashboard generates synthetic transactions, engineers features in real time, and displays the fraud verdict.

Synthetic Generator → Feature Engineering → LightGBM Model → Fraud Verdict

Live Demo →

Demo runs on a separate machine — Docker Compose with Flask API (port 5050) + Next.js UI (port 3000)